

پروژه فاز 3 انتقال مطمئن

زهره کریمی

آرمان اوشائی

1. مقدمه و هدف پروژه

هدف از این فاز پروژه، پیاده‌سازی پروتکل انتقال مطمئن داده (Reliable Data Transfer) نسخه 3.0 بر روی پروتکل لایه انتقال غیرمطمئن UDP است. همان‌طور که می‌دانیم، UDP سرویسی "غیرمتصل" (Connectionless) و بدون تضمین تحویل است که ممکن است بسته‌ها را گم کرده (Loss) یا داده‌های آن‌ها را مخدوش (Corrupt) کند. در این پروژه، ما با پیاده‌سازی مکانیزم‌های ACK، Checksum، Sequence Number و Timer، یک کانال ارتباطی مطمئن (Reliable Channel) را به صورت نرم‌افزاری شبیه‌سازی کردیم. الگوریتم استفاده شده در این پروژه Stop-and-Wait می‌باشد.

2. ساختار ماژولار برنامه

کد پروژه به چهار ماژول اصلی تقسیم شده است:

الف) ماژول `utils.py` بسته‌بندی و بررسی خطا

این ماژول وظیفه ساختاردهی به بسته‌ها (Packet Structure) را بر عهده دارد.

- هدر بسته: طبق مستندات، هدر شامل فیلدهای (Type داده یا ACK، Sequence Number، Length) و Checksum است.
- محاسبه Checksum: تابع `calculate_checksum` بدون استفاده از کتابخانه‌های آماده پیاده‌سازی شده است. این تابع تمام بایت‌های پیام را به صورت کلمات ۱۶ بیتی جمع کرده و متمم یک (One's Complement) آن را محاسبه می‌کند تا هرگونه تغییر بیت در مسیر تشخیص داده شود.

ب) ماژول `network.py` شبیه‌سازی کانال غیرمطمئن

از آنجا که شبکه‌های محلی (Loopback) معمولاً خطا ندارند، این ماژول وظیفه تزریق خطای مصنوعی را دارد. تابع `simulate_channel` با احتمالات مشخص (PROB_DROP) و (PROB_CORRUPT) تصمیم می‌گیرد که:

1. بسته را کاملاً حذف کند (شبیه‌سازی Packet Loss).
2. بیت‌هایی از بسته را تغییر دهد (شبیه‌سازی Bit Error/Corruption).

ج) ماژول `rdt.py` هسته پروتکل

این کلاس شامل ماشین حالت محدود (FSM) فرستنده و گیرنده است.

- **متد send:** داده را بسته‌بندی کرده و ارسال می‌کند. سپس وارد یک حلقه انتظار برای دریافت ACK می‌شود. اگر در زمان مقرر (Timeout) پاسخی نرسد یا پاسخ خراب باشد، بسته مجدداً ارسال می‌شود. (Retransmission)
- **متد recv:** دائماً سوکت را گوش می‌دهد. با دریافت بسته، ابتدا Checksum را بررسی می‌کند. اگر سالم بود و شماره توالی (Seq) صحیح بود، ACK مربوطه را ارسال کرده و داده را تحویل می‌دهد. اگر بسته تکراری بود (Duplicate)، مجدداً ACK قبلی را ارسال می‌کند.

د) ماثول main.py لایه کاربرد

این فایل نقش کلاینت و سرور را بازی می‌کند. سرور روی پورت مشخصی گوش می‌دهد و کلاینت رشته‌های متنی را ارسال می‌کند تا عملکرد صحیح پروتکل تست شود.

3. نتیجه‌گیری

در این پروژه نشان داده شد که چگونه می‌توان با افزودن منطق نرم‌افزاری در لایه کاربرد/انتقال، یک سرویس کاملاً غیرقابل اعتماد (UDP) را به یک سرویس مطمئن تبدیل کرد. پروتکل RDT 3.0 با موفقیت توانست در برابر خطاهای Loss و Corruption مقاومت کند و تمام داده‌ها را به ترتیب و بدون خطا به مقصد برساند.

4. نحوه اجرای کد در ترمینال

1 => Python main.py server

2 => python mian.py client

```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS D:\Network\RDT 3.0> python main.py server
--- SERVER STARTED ---
Listening on 12000...
[Recv] Packet 0 OK. Sending ACK.
>>> Server delivered data: Hello RDT
[Recv] Packet 1 OK. Sending ACK.
>>> Server delivered data: Packet 2
[Recv] Packet 0 OK. Sending ACK.
>>> Server delivered data: Packet 3
[Recv] Packet 1 OK. Sending ACK.
>> Network: Packet CORRUPTED!
>>> Server delivered data: Final Packet
[Recv] Duplicate Packet 1. Resending ACK 1.

PS D:\Network\RDT 3.0> python main.py client
--- CLIENT STARTED ---

--- Sending: Hello RDT ---
[Send] Seq=0 sent. Waiting for ACK...
[Send] ACK 0 received correctly. Moving on.

--- Sending: Packet 2 ---
[Send] Seq=1 sent. Waiting for ACK...
[Send] ACK 1 received correctly. Moving on.

--- Sending: Packet 3 ---
[Send] Seq=0 sent. Waiting for ACK...
[Send] ACK 0 received correctly. Moving on.

--- Sending: Final Packet ---
[Send] Seq=1 sent. Waiting for ACK...
[Send] Garbage or Wrong ACK received. Ignore.
[Send] Seq=1 sent. Waiting for ACK...
[Send] ACK 1 received correctly. Moving on.
  
```

5. پاسخ به سوالات

چرا به RDT نیاز داریم وقتی UDP خودش Checksum دارد؟

UDP فقط خرابی بیت‌ها را با Checksum تشخیص می‌دهد، اما هیچ تضمینی برای گم نشدن بسته، ترتیب تحویل، جلوگیری از تکرار، یا ارسال مجدد ندارد. RDT این کمبودها را با Sequence Number، ACK، و Timeout جبران می‌کند و UDP را به یک سرویس قابل اعتماد تبدیل می‌کند.

تفاوت‌های اصلی RDT 3.0 و TCP چیست؟

RDT 3.0 از Stop-and-Wait استفاده می‌کند و در هر لحظه فقط یک بسته در پرواز دارد، اما TCP از Sliding Window، کنترل ازدحام، کنترل جریان و Handshake پیچیده استفاده می‌کند. RDT ساده و آموزشی است، TCP کامل و صنعتی.

در پیاده‌سازی RDT شما، هر سگمنت شامل چه فیلدهایی است و چرا؟

هر سگمنت شامل Type، Sequence Number، Length، Checksum و Payload است. Type برای تشخیص Sequence Number، DATA/ACK برای تشخیص تکرار و ترتیب، Length برای استخراج درست داده، Checksum برای تشخیص Corruption، و Payload برای حمل داده کاربردی استفاده می‌شود.

اگر یک بسته داده (DATA) گم شود، چه اتفاقی می‌افتد؟

گیرنده هیچ بسته‌ای دریافت نمی‌کند، بنابراین ACK ارسال نمی‌شود. فرستنده پس از انقضای Timeout متوجه Loss می‌شود و همان بسته را دوباره ارسال می‌کند.

اگر یک بسته ACK گم شود، چه تفاوتی با گم شدن DATA دارد؟

در گم شدن ACK، گیرنده داده را قبلاً تحویل داده است، اما فرستنده ACK را نمی‌گیرد و Timeout می‌شود، بنابراین بسته DATA را دوباره می‌فرستد. گیرنده با دیدن Sequence Number تکراری، داده را دوباره تحویل نمی‌دهد و فقط ACK قبلی را ارسال می‌کند.

6. لینک ویدئوهای ضبط شده به عنوان ارائه